Generative Modelling Case Study Report

Assignment: Generative modelling case study (40%); **Module:** Advanced Research Topic in Data Science

Name: Deepak Raj Manickam
Student ID: 23070139; **Student Email ID:** dm24aav@herts.ac.uk
GitHub Repository: https://github.com/deepakraj-04/Generative-Modelling-Case-Study

Introduction:

Generative Adversarial Networks (GANs), introduced by Ian Goodfellow et al. in 2014, are a class of deep learning models used to generate new data that resembles a training dataset. A GAN contains two neural networks trained together. The generator creates fake samples from random noise, while the discriminator attempts to decide whether a sample is real or generated. As training continues, the generator improves by learning from the discriminator's feedback.

GANs have been used in image generation, medical data augmentation, synthetic traffic simulation, and creative tasks such as sketch generation. However, training can be unstable, and results often depend on the dataset, model design, and training settings.

This report evaluates GAN performance across both simple and practical tasks. Part 1 focuses on two-dimensional synthetic data to understand model behavior clearly. Part 2 applies GANs to blood-cell image generation, cybersecurity traffic generation, and hand-drawn pizza sketch generation.

Part 1: Building and Understanding GANs from Scratch

Sine-Wave Distribution:

The first experiment reproduced a GAN trained on a noisy sine-wave dataset. Two-dimensional coordinates were generated where:

- x-values were sampled uniformly from a fixed range
- y-values followed a sine relationship with added Gaussian noise

This task was useful because the true pattern is easy to visualize, making it simple to judge whether the generator has learned correctly.

At the start of training, generated samples were random and scattered. After training for 800 epochs, the generator learned the overall sine-wave structure and placed most generated points close to the real curve.

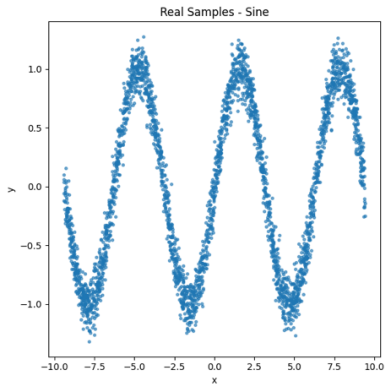
Final baseline losses were:

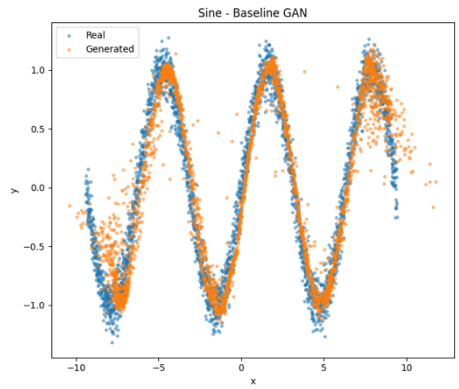
- Generator loss = 0.6971
- Discriminator loss = 1.3875

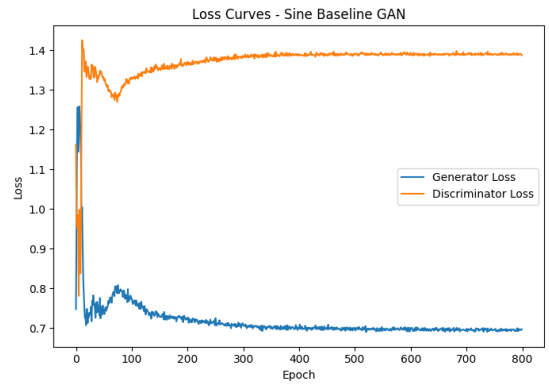
Final modified model losses were:

- Generator loss = 0.6979
- Discriminator loss = 1.3861

Both models performed well. The modified version gave slightly smoother outputs, although the difference was small.







The loss values fluctuated during training, which is common in GANs because both networks are learning at the same time.

Spiral Distribution:

To test the GAN on a more difficult shape, a noisy spiral dataset was selected. The spiral is harder than the sine-wave because the generator must learn a turning structure rather than one smooth curve.

After 800 epochs, the model captured the main spiral form but showed uneven density. Some areas had too many points, while others were sparse.

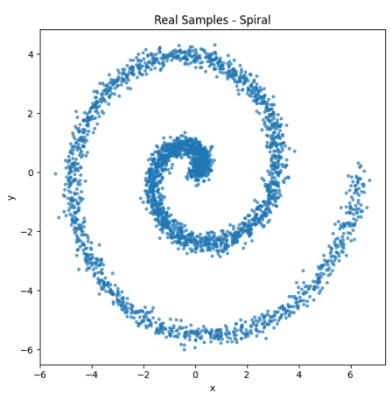
Final baseline losses were:

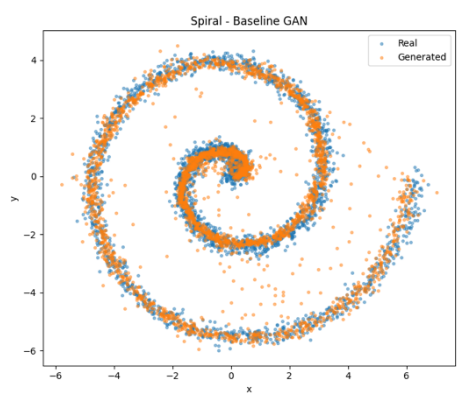
- Generator loss = 0.7077
- Discriminator loss = 1.3803

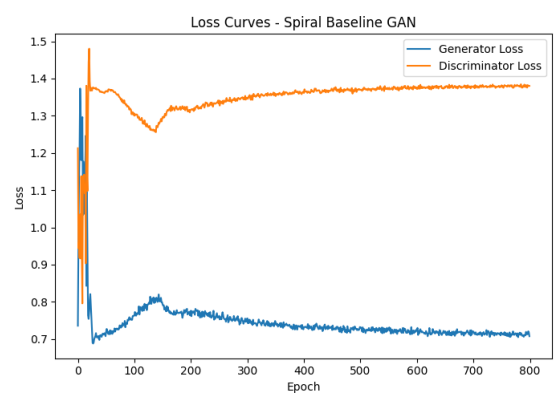
Final modified model losses were:

- Generator loss = 0.6926
- Discriminator loss = 1.3867

The modified model performed better on this task, especially in the outer sections of the spiral.







This shows that GAN difficulty increases when the target structure becomes more complex.

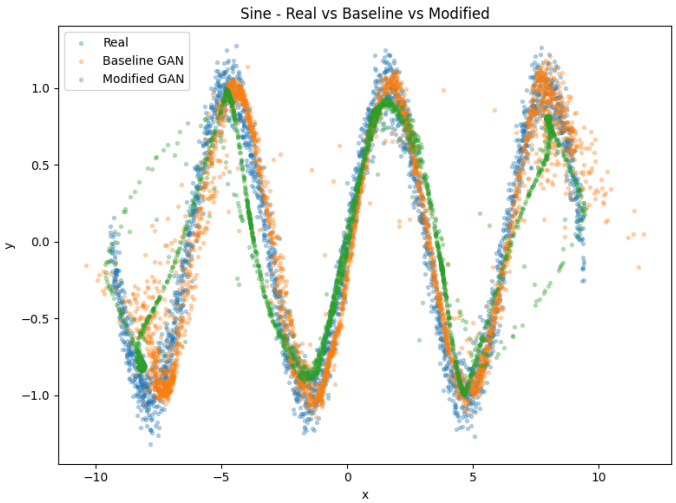
Architecture Modification:

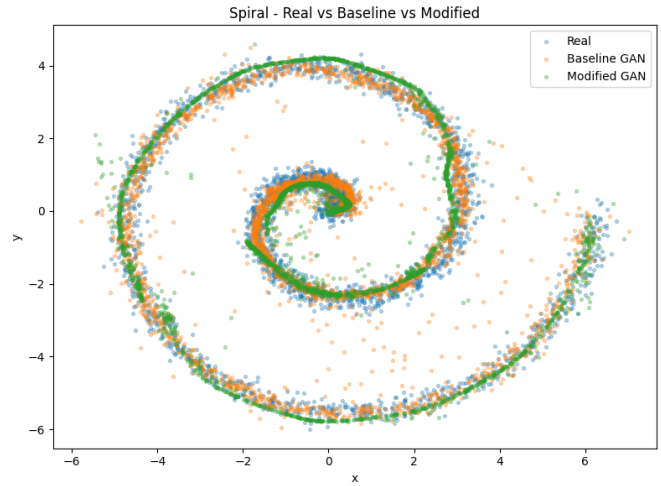
A second GAN architecture was tested using:

- deeper hidden layers

- Tanh activations in the generator
- slightly higher model capacity

Compared with the baseline model, the modified GAN produced cleaner and more stable results on the spiral dataset. On the sine-wave task, both versions were strong.





The results suggest that moderate increases in model depth can help when learning more difficult shapes.

Part 2.1: Blood Cell Microscope Imaging (BloodMNIST)

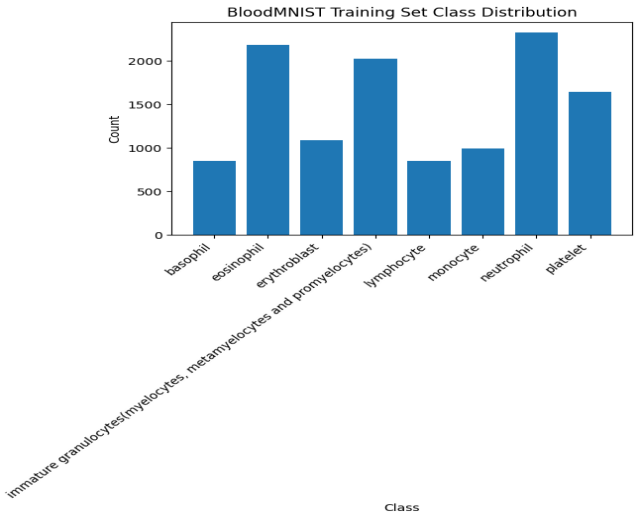
Dataset Exploration:

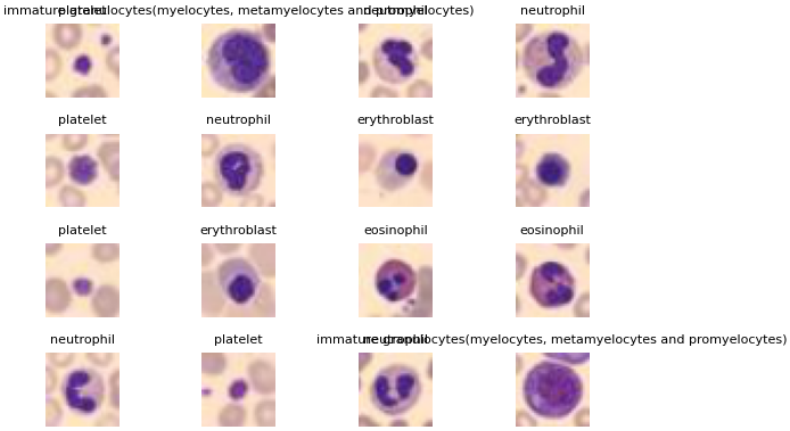
The BloodMNIST dataset contains labelled blood-cell microscope images across eight classes. The dataset sizes used were:

- Training set = 11,959 images
- Validation set = 1,712 images
- Test set = 3,421 images

The classes include basophils, eosinophils, erythroblasts, immature granulocytes, lymphocytes, monocytes, neutrophils, and platelets.

Sample inspection showed clear differences in colour, texture, and cell shape.





DCGAN Model:

Because the data are images, a Deep Convolutional GAN (DCGAN) was used. The architecture contained:

- transposed convolution layers in the generator
- convolution layers in the discriminator
- batch normalisation
- ReLU and LeakyReLU activations

This structure is suitable for images because it can learn local patterns such as shapes and texture.

Results and Discussion:

The model was trained for 50 epochs.

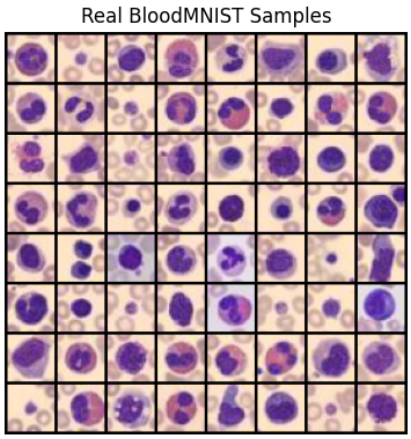
At epoch 1:

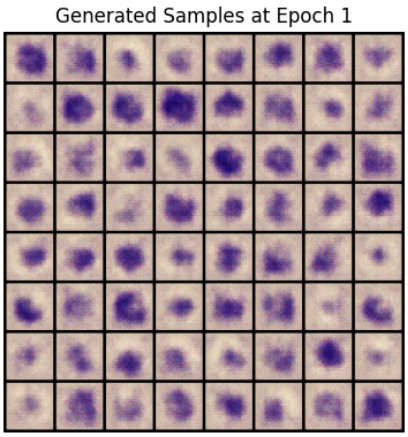
- Generator loss = 4.6132
- Discriminator loss = 0.4400

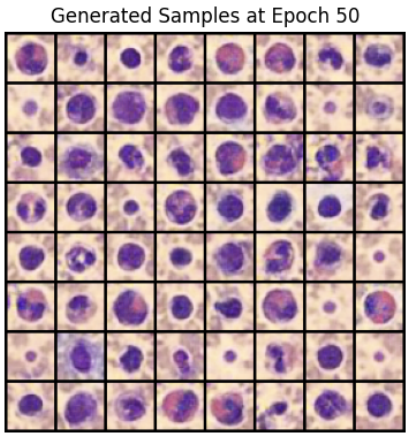
At epoch 50:

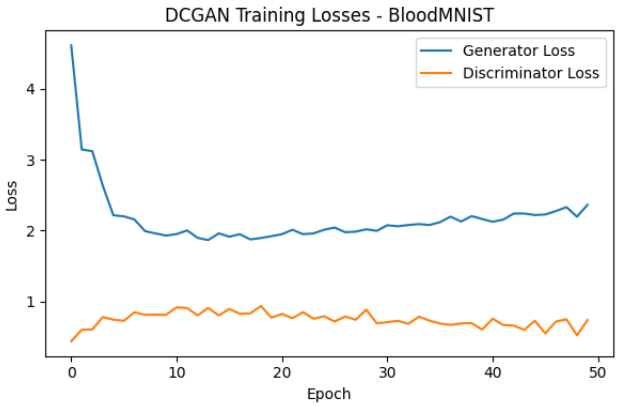
- Generator loss = 2.3640
- Discriminator loss = 0.7383

Early outputs were noisy and unclear. After further training, generated images began to resemble microscope imagery, with visible circular shapes and realistic color tones.









A Fréchet Inception Distance (FID) score was attempted, but the required package was unavailable in the runtime environment.

Although some generated images were convincing, many remained blurred or lacked fine detail. This is understandable given the small image resolution and limited training time.

GAN-generated blood-cell images could be useful for:

- increasing small training sets

- supporting rare classes
- creating synthetic examples for model testing

However, synthetic biomedical images should always be checked carefully before practical use.

The same method could also be used to generate ducks or other natural image categories, although a more varied dataset would likely need longer training.

Part 2.2: Cybersecurity – Synthetic Traffic (CICIDS 2017)

Dataset Preparation:

The Wednesday subset of the CICIDS 2017 dataset was used. It contained:

- BENIGN traffic
- DoS / DDoS attack traffic

Unlike image tasks, this dataset contains numerical network-flow features.

Original dataset size:

- 243,389 rows × 79 columns

After filtering and cleaning:

- 242,679 rows

Final usable numeric features:

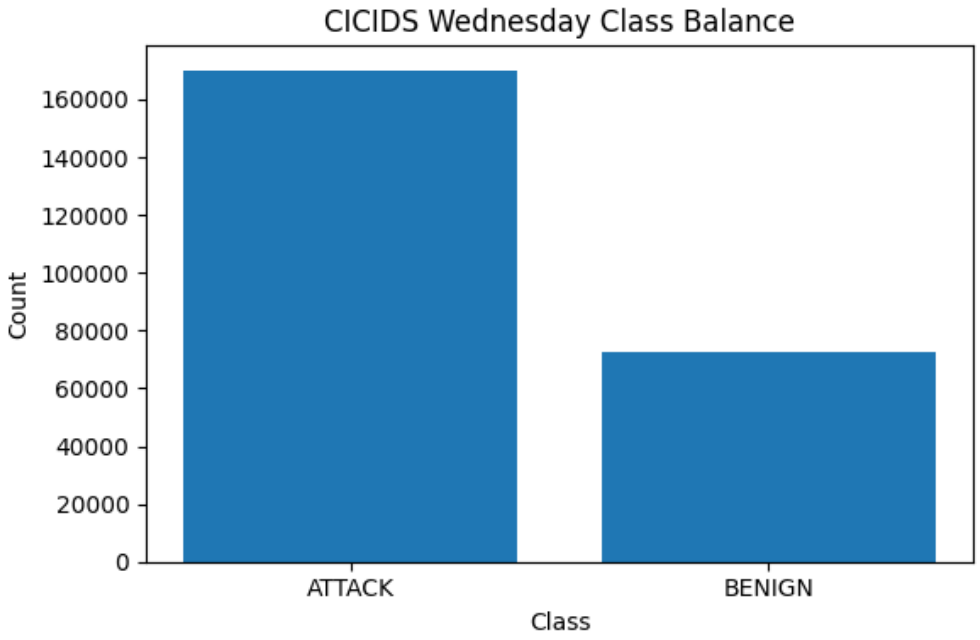
- 68 features

Class counts:

- ATTACK = 170,064
- BENIGN = 72,615

Data cleaning included:

- removing missing values
- removing infinite values
- filtering selected classes
- feature scaling



Tabular GAN:

A fully connected GAN was used because tabular traffic data do not have image structure. The generator produced synthetic feature rows, while the discriminator classified rows as real or fake.

Results and Discussion:

The model was trained for 80 epochs.

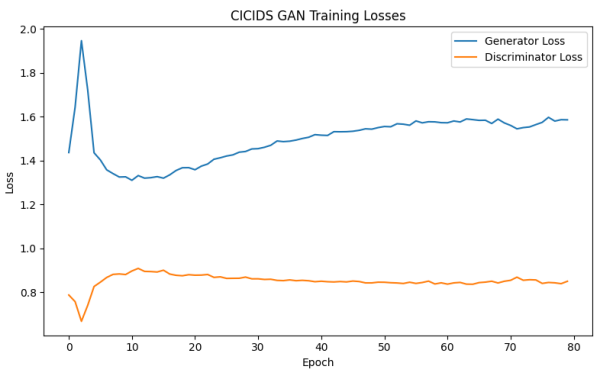
At epoch 1:

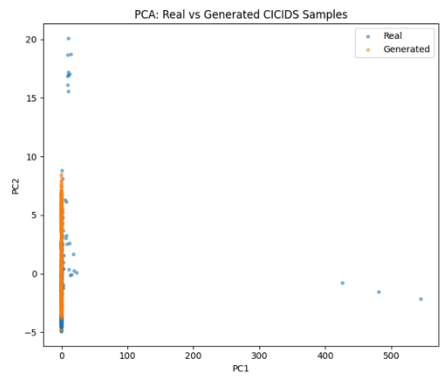
- Generator loss = 1.4365
- Discriminator loss = 0.7877

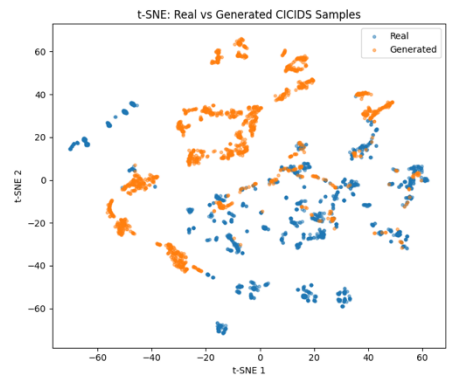
At epoch 80:

- Generator loss = 1.5856
- Discriminator loss = 0.8502

Training losses became more stable after the early stages.







To compare real and synthetic data, PCA and t-SNE were used.

The generated data overlapped partly with the real distribution, which suggests that the GAN learned some overall structure. However, differences in clustering were still visible.

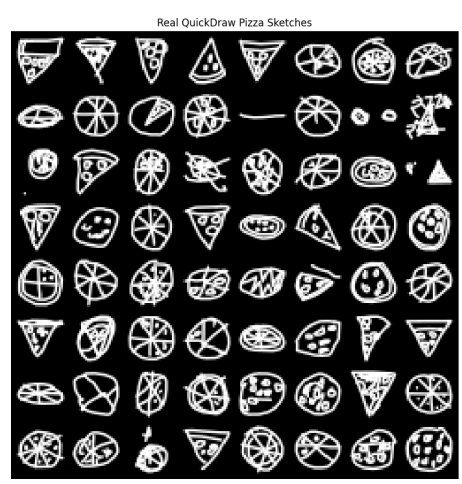
Feature comparisons also showed mixed results. Some variables were close to the original data, while others were not.

Synthetic network traffic can support:

- intrusion detection training
- safe data sharing
- simulation environments

However, tabular GAN generation is difficult because many traffic features depend strongly on each other.

Part 2.3: Creative AI – QuickDraw Pizza



Dataset Overview:

The QuickDraw pizza subset contains hand-drawn pizza sketches.

Dataset size used:

- 30,000 images

Unlike BloodMNIST, the goal here was not photo realism, but recognisable sketch generation.

DCGAN Training:

A grayscale DCGAN was trained on 28×28 pizza sketches for 50 epochs.

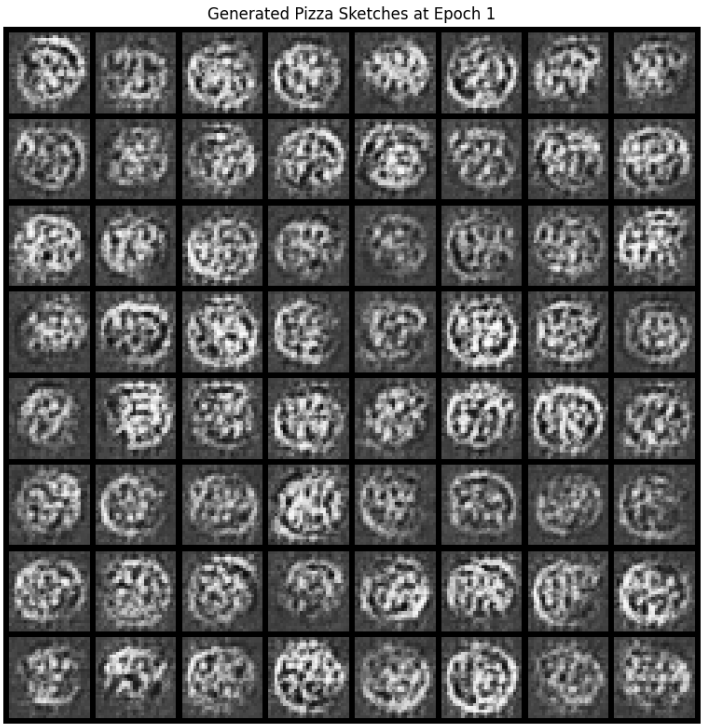
At epoch 1:

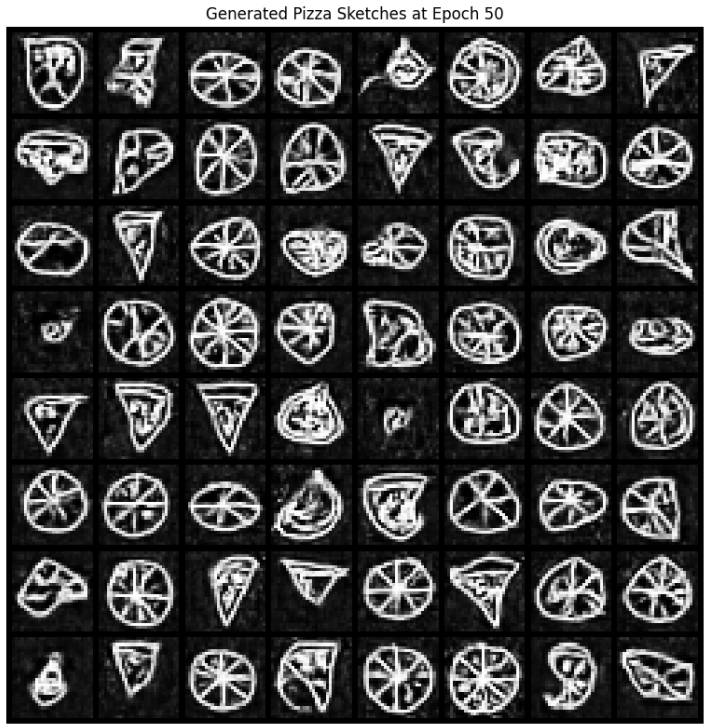
- Generator loss = 3.6950
- Discriminator loss = 0.4326

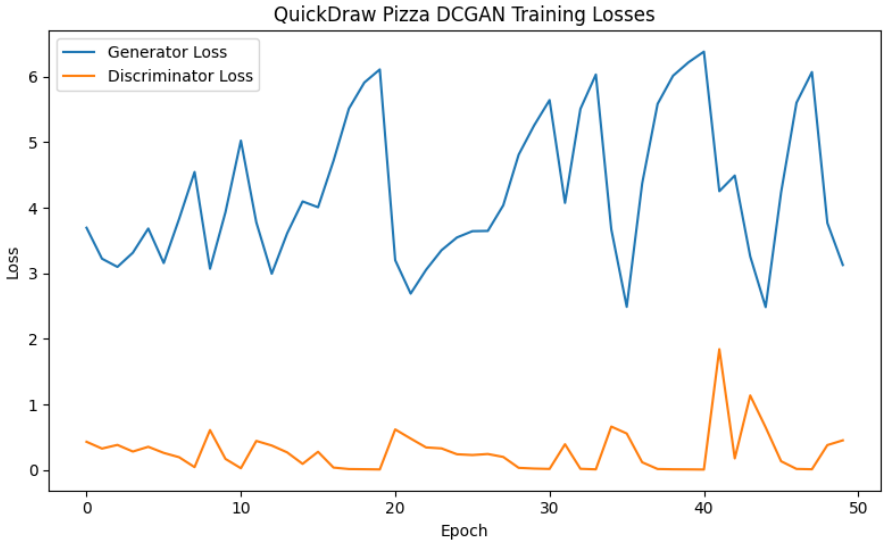
At epoch 50:

- Generator loss = 3.1280
- Discriminator loss = 0.4546

Early outputs looked random, but later epochs produced clearer circular pizza-like shapes and topping marks.







Results and Discussion:

Many final outputs were recognisably pizza-like, although some were incomplete or distorted. Compared with BloodMNIST, this task was easier because sketches require less detail than natural or medical images.

The model therefore performed reasonably well on a simpler target category.

Overall Comparison:

Task	Data Type	Model Type	Difficulty	Quality of Results
Sine-Wave	2D points	Fully Connected GAN	Low	Strong
Spiral	2D points	Fully Connected GAN	Medium	Moderate
BloodMNIST	RGB Images	DCGAN	High	Moderate
CICIDS	Tabular Data	Fully Connected GAN	High	Moderate
QuickDraw Pizza	Sketch Images	DCGAN	Medium	Good

Conclusion:

This assignment showed how GANs can be adapted to very different kinds of data. In Part 1, synthetic 2D datasets gave a clear view of training behaviour. The sine-wave task was learned well, while the spiral task showed the need for stronger model capacity.

In Part 2, image-based DCGANs produced useful outputs for both blood-cell images and sketch generation, while a tabular GAN generated partly realistic cybersecurity traffic samples. Across all tasks, common issues included instability, blur, and uneven diversity.

Overall, the experiments show that GANs are effective tools for synthetic data generation when the model design matches the type of dataset being used.

References:

GANs Documentation

MedMNIST Dataset documentation.

CICIDS 2017 Dataset documentation.

QuickDraw Dataset.